

Agentic Test Generation for BOLA/IDOR Detection in REST APIs

Valérie Jacobs

Supervisor: Prof. Serge Demeyer & Tolgahan Bardakci

Master's Program in Data Science and AI
University of Antwerp

July 26, 2026

1 Problem Statement

Large Language Models (LLMs) have become increasingly more capable of generating high-quality test cases [1] [2] [3]. However, they often lack systematic methods for identifying complex authorization vulnerabilities, such as Broken Object Level Authorization (BOLA)/ Insecure Direct Object References (IDOR). Instead, they mainly rely on brute-force testing strategies that fail to understand dependencies between request parameters, resulting in a large number of invalid requests. Consequently, many security vulnerabilities are being discovered incidentally, rather than through targeted, context-aware testing [12] [14].

As a result, authorization vulnerabilities such as BOLA/ IDOR remain among the most common security vulnerabilities in Representational State Transfer (REST) Application Programming Interfaces (APIs) [7][14]. This research aims to address this gap, leading to the following research questions:

RQ1: To what extent can an agentic LLM planner improve the discovery of BOLA/IDOR vulnerabilities compared to [the most relevant methods detecting BOLA: I'm still working on this

- Static and Dynamic Application Security Testing (SAST/DAST), are ineffective at finding BOLAs [15].
- Links2CPN: requires JSON-formatted server logs as inputs. The selected GitHub projects did not include such logging methods. The only possibility would be to implement such logging techniques manually into the projects → probably too time-consuming

- Static analysis of API spec Barabanov et al [16]: tool is not open-sourced, and the authors are impossible to contact according to [15]
- Palo Alto Networks' "BOLABuster": Blog post does not entail either the prompt design or the specific LLMs used and does not provide any empirical evidence to substantiate their findings.
- Other frameworks exist, stateful fuzzers (RESTler, 42Crunch's audit tool, DeepREST, NAUTILUS). But they do not specifically target BOLAs. Moreover, traditional methodologies like fuzzing and static analysis are said to be ineffective in detecting BOLAs, making manual detection the standard approach [15].

I have not yet looked into:

- AuthProbe: Specification-Driven, Multi-Identity Detection of Broken Object-Level Authorization in Recruitment APIs
- BOLArray
- BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications
- KARTAL: Web Application Vulnerability Hunting Using Large Language Models
- BACFuzz: Grey-box Fuzzer
- Karate-BOLA: LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience

], and what are the associated costs?

RQ2: How does the quality of discovered violations change with API complexity?

RQ3: Do LLM-generated authorization-violation rationales correlate with ground-truth violations ?

2 Relevancy

REST APIs form the backbone of modern web applications, cloud services, and mobile applications [6]. Ensuring that authorization is correctly enforced is therefore very important. According to the OWASP API Security Top 10 (2023), Broken Object Level Authorization (BOLA) has consistently been ranked as the most critical API security vulnerability [7].

Here there is supposed to be a paragraph about the current techniques but I'm still actively looking into these.

This research aims to address this limitation by developing an agentic AI system that analyzes an OpenAPI specification and generates security-focused, multi-step authorization tests that reason about multiple users, object ownership, and resource lifecycles. So, instead of executing isolated requests, the proposed system will reason about sequences of API interactions involving multiple identities and object states to uncover authorization flaws. Besides contributing to software testing research, such a system could assist developers in finding security vulnerabilities earlier during development while producing reproducible and explainable test cases.

3 Why Can I Solve This?

Throughout my studies I have developed an interest in cybersecurity, software testing, and AI. I have practical experience developing REST APIs and software systems, which allows me to understand both the implementation and testing perspectives.

In addition, my academic background has provided me with knowledge of software testing, software quality, and machine learning techniques. This combination makes me well suited to investigate how agentic LLMs can be applied to automated security testing.

The project also aligns with the expertise of the AnSyMo research group, allowing me to build upon existing research on REST API testing and LLM-based amplification and also extending it towards vulnerability-oriented testing.

4 How to Solve the Problem?

The first step is to analyze the API specification to identify resources, relationships, and ownership information. This will be stored in the agent's context window.

The core of the system will be an agentic LLM planner that generates sequences of API requests. Rather than testing single endpoints independently, the planner will reason about object creation, ownership transfer, lifecycle changes, and access attempts performed by different users.

The generated requests will then be executed against the target API, after which the responses will be analyzed to detect potential authorization violations.

Finally, any discovered vulnerabilities will be written to a report.

5 Evaluation

5.1 Evaluation of RQ1: Detection Effectiveness and Cost

Metrics to compare with are not yet set in stone, because I haven’t identified which methods I best compare with as I previously stated in the bullet points in the statement of RQ1. Although these metrics seem to occur frequently.

The proposed approach will be evaluated using intentionally vulnerable benchmark REST APIs containing known BOLA’s. In this paper 12 intentionally vulnerable API’s with known BOLA’s are open-sourced, so these might be good to use [15].

$$\text{Precision} = \frac{TP}{TP+FP}$$

Where TP denotes correctly identified BOLA vulnerabilities and FP denotes incorrectly reported authorization violations. This metric has previously been used in the evaluation of LLM-based penetration testing systems such as Pen-testGPT.

Recall measures the proportion of known BOLA vulnerabilities that are successfully detected:

$$\text{Recall} = \frac{\text{Detected BOLA vulnerabilities}}{\text{Total known BOLA vulnerabilities}}$$

Since the benchmark APIs contain documented vulnerabilities, recall can be computed by comparing detected vulnerabilities against the known ground truth.

Besides detection effectiveness, the computational cost of the approach will be measured. Following previous work such as RESTler and Karate-BOLA-Guard, the following efficiency metrics will be collected:

- Number of API requests required per confirmed vulnerability
- Number of LLM tokens consumed per confirmed vulnerability
- Total wall-clock execution time until a confirmed vulnerability is discovered

5.2 Evaluation of RQ2: The effect of API complexity

The second research question investigates whether the proposed approach remains effective for APIs with varying complexity. Detection performance will be analyzed with respect to API complexity. To evaluate this, two tools are used to characterize the APIs under test: RAMA and OASQuali. RAMA is used to quantify the structural complexity of an API, while OASQuali is used to assess the quality of its OpenAPI specification.

[**Note to mentor:** The RAMA cli has been tested on all 12 vulnerable API's and it works on all of them.]

RAMA: The RAMA CLI provides a set of metrics for measuring API complexity automatically.

- **Average Path Length (APL):** Average length of API paths [18] [19].
Average path length captures the overall structural depth of the API. Longer paths generally indicate more complex resource hierarchies and potentially more complex authorization relationships.
- **Arguments per Operation (APO):** Average amount of operation arguments for a service [18] [19].
Operations requiring more parameters provide more possible combinations of identifiers that an agent must handle when testing authorization.
- **Biggest Root Coverage (BRC):** Measures how much of the API is reachable from its largest root resource.
A high BRC indicates that a substantial portion of the API is organized around a common resource. So a low BRC, meaning the resource space is more scattered, can increase the complexity of relationships that an agent must explore.
- **Data Weight (DW):** Represents the complexity of the data types used in input and output messages; the amount/size of data structure. It is computed by counting all path parameters and all parameters in request and response bodies within all operations.
More complex request and response structures can increase the amount of information the agent must process when constructing authorization tests.
- **Distinct Message Ratio (DMR):** Captures the complexity/cohesion of data types in an interface; it measures the uniqueness of the structures. It is computed by dividing the count of distinct messages that occur only once in the whole API by the number of all messages.
A higher DMR suggests that the agent will encounter a wider variety of request and response formats, increasing the reasoning effort required.
- **Number of Roots (NOR):** Measures the number of independent root resources in the API. A root resource is any resource that has no incoming links [17] [18].
A larger number of roots indicates a more fragmented API structure, requiring the agent to explore more resource hierarchies.
- **Longest Path (LP):** Referring to the length of the longest path of resources connected by links. In other words, the path length of a resource is defined by the number of slashes (“/”) in its path[17] [18].

A high maximum depth indicates that at least some operations require the agent to reason across a long sequence of resource relationships. This can make it more difficult to construct valid requests and maintain the necessary context when testing authorization boundaries.

- **Lack of Message-Level Cohesion (LoC_msg):** Measures how dissimilar the input and output data of operations within an interface are. A higher lack of cohesion requires the agent to handle a wider range of request and response structures.
- **Service Interface Data Cohesion (SIDC):** Measures how strongly operations within a service share input parameters and return types. A higher lack of cohesion requires the agent to handle a wider range of input parameter and return type structures.
- **Weighted Service Interface Count (WSIC):** Measures the size of an interface by considering the number of exposed operations. A larger interface provides the agent with more operations to explore and more potential interactions through which authorization violations may occur.

Thresholds for these metrics were calculated from a dataset of 1,737 API description files. [18] These thresholds can therefore be used as reference points for determining the relative complexity of the APIs included in the evaluation.

[**Note to mentor:** I have tested OASQuali on a few of the 12 vulnerable API's. However the tool currently only supports JSON (and even with some JSON's I get what seems to be a syntax error).

More than half of the API's is written in YAML. An option is to convert JSON to YAML although it is possible that this can not happen losslessly if certain elements are used in the API, I have not yet checked for this.

Another option is to contribute to the repo and add YAML support, but I cannot foresee how much work this would be (<https://github.com/alixdecr/oas-quality-tool>).]

OASQuali: API complexity alone does not determine how difficult an API is for an agent to test. The quality and completeness of the OpenAPI specification also influence how much reliable information is available for understanding the API and generating test cases. OASQuali provides a deterministic assessment of OpenAPI documentation quality across five dimensions and 17 concrete evaluations [20]. The Administrative Governance and Operational Readiness dimensions will be omitted because of irrelevance.

- **Structural Integrity:** This dimension consists of *validate-json* and *validate-oas*, which check whether the specification is valid JSON and conforms to the OpenAPI metamodel [20]. This is relevant because the specification

serves as the agent’s input for understanding the API. Structural errors can prevent reliable parsing reducing the agent’s ability to generate valid BOLA/IDOR tests.

- **Evolutionary Maturity:** This dimension consists of *oas-version*, which checks whether the specification uses OAS version 3.0.0 or later [20]. This is relevant because newer OAS versions provide support for modern specification features. A more up-to-date specification can therefore provide the agent with a more consistent representation of the API, whereas older specifications may limit the information available.
- **Semantic Communicability:** This dimension consists of *route-descriptions*, *response-descriptions*, *parameter-descriptions*, *response-examples*, and *parameter-examples* [20]. These metrics measure how much semantic information is available to the agent. Route and parameter descriptions can clarify the purpose of resources and identifiers, while examples provide concrete information about how requests and responses are structured. Poor semantic communicability can therefore increase the amount of effort required from the agent when reasoning about authorization violations.

Overall, OASQuali complements the RAMA metrics by measuring a different property of the API under test. RAMA characterizes the structural and interface complexity of the API, whereas OASQuali characterizes the quality of the machine-readable specification used by the agent.

5.3 Evaluation of RQ3: Quality of Generated Reasoning

The third research question evaluates whether the report generated by the LLM accurately describes the underlying authorization violations.

Each detected vulnerability will be written to a report. These explanations will be compared against the known ground truth to determine whether the agent correctly identifies the violated authorization assumptions.

The quality of the generated test scenarios will be evaluated using execution-based metrics. The executable scenario rate is defined as:

$$\text{Executable scenario rate} = \frac{\text{Executable generated scenarios}}{\text{All generated scenarios}}$$

Which measures how often the generated API interaction sequences can be executed successfully without manual correction.

Finally, completeness will be assessed by measuring whether the generated scenarios include all required steps of a successful authorization attack, including object creation, ownership establishment, authentication with multiple identities, unauthorized access attempts, and verification of the obtained responses.

6 Timeline

- **July-August:** Conduct a literature review on REST API testing, authorization vulnerabilities, OpenAPI specifications, and agentic LLMs. Define the research scope.
- **August:** Implement the OpenAPI parser and automatically create multiple testing identities. Design the initial architecture.
- **September-December:** Develop the agentic LLM planner capable of generating multi-step authorization test scenarios. Integrate the execution framework.
- **January-March:** Implement vulnerability detection, execute experiments, and compare the proposed approach with existing techniques.
- **April-May:** Analyze the experimental results and write the master’s thesis.
- **June:** Finalize the thesis, prepare the presentation, and defend the research.

References

- [1] Tasarsu, M., Tokmak, A. V., & Catal, C. (2026). Test case generation using large language models: A systematic literature review. *Cluster Computing*, 29(4), 227. <https://doi.org/10.1007/s10586-026-06021-z>
- [2] Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering*, 50(1), 85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- [3] Zhang, Q., Fang, C., Gu, S., Shang, Y., Chen, Z., & Xiao, L. (2025). Large Language Models for Unit Testing: A Systematic Literature Review (arXiv:2506.15227). *arXiv*. <https://doi.org/10.48550/arXiv.2506.15227>
- [4] Jayaraman, B., Marathe, V. J., Mozaffari, H., Shen, W. F., & Kenthapadi, K. (n.d.). *Permissioned LLMs: Enforcing Access Control in Large Language Models*.
- [5] Li, Y., Liu, Y., He, X., Zhao, Y., Li, Q., & Xu, K. (2026). Chain-of-Authorization: Embedding authorization into large language models (arXiv:2603.22869). *arXiv*. <https://doi.org/10.48550/arXiv.2603.22869>
- [6] Peldszus, S., Rutenkolk, J., Heide, M., Sollmann, J., Klatt, B., Köhne, F., & Berger, T. (2026). Developer Perspectives on REST API Usability: A Study of REST API Guidelines (arXiv:2601.16705). *arXiv*. <https://doi.org/10.48550/arXiv.2601.16705>

- [7] OWASP API Security Project — OWASP Foundation. (n.d.). Retrieved July 26, 2026, from <https://owasp.org/www-project-api-security/>
- [8] Atlidakis, V., Godefroid, P., & Polishchuk, M. (2019). RESTler: Stateful REST API Fuzzing. 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 748–758. <https://arxiv.org/abs/1806.09739>
- [9] Martín López, A., Segura Rueda, S., & Ruiz Cortés, A. (2021). RESTTest: Automated black-box testing of RESTful web APIs. Association for Computing Machinery (ACM). <https://doi.org/10.1145/3460319.3469082>
- [10] Dharmaadi, I. P. A., Alhanahnah, M., Pham, V.-T., Mohsen, F., & Turkmen, F. (2025). BACFuzz: Exposing the Silence on Broken Access Control Vulnerabilities in Web Applications (arXiv:2507.15984). arXiv. <https://doi.org/10.48550/arXiv.2507.15984>
- [11] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. 2022 IEEE Symposium on Security and Privacy (SP), 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [12] Marian Pasca, E., Delinschi, D., Erdei, R., & Matei, O. (2025). LLM-Driven, Self-Improving Framework for Security Test Automation: Leveraging Karate DSL for Augmented API Resilience. IEEE Access, 13, 56861–56886. <https://doi.org/10.1109/ACCESS.2025.3554960>
- [13] Kaur, B. (2026). Broken Object Level Authorization in the Wild: An Empirical Taxonomy from 100+ Bug Bounty Disclosures (arXiv:2605.25865). arXiv. <https://doi.org/10.48550/arXiv.2605.25865>
- [14] Tanveer, F., Iradat, F., Iqbal, W., & Ahmad, A. (2025). Towards Secure APIs: A Survey on RESTful API Vulnerability Detection. Computers, Materials & Continua, 84(3), 4223–4257. <https://doi.org/10.32604/cmc.2025.067536>
- [15] Johansens, E. (2025). Detecting BOLA Vulnerabilities with Large Language Models.
- [16] Bauman Moscow State Technical University, Barabanov, A., Dergunov, D., Advanced Software Technology Laboratory, Huawei, Makrushin, D., Chong-Ming Technology Center, Huawei, Teplov, A., & Advanced Software Technology Laboratory, Huawei. (2022). AUTOMATIC DETECTION OF ACCESS CONTROL VULNERABILITIES VIA API SPECIFICATION PROCESSING. Voprosy Kiberbezopasnosti, (1(47)), 49–65. <https://doi.org/10.21681/2311-3456-2022-1-49-65>
- [17] Haupt, F., Leymann, F., & Vukojevic-Haupt, K. (2018). API governance support through the structural analysis of REST APIs. Computer Science - Research and Development, 33(3–4), 291–303. <https://doi.org/10.1007/s00450-017-0384-1>

- [18] Bogner, J., Wagner, S., & Zimmermann, A. (2020). Collecting Service-Based Maintainability Metrics from RESTful API Descriptions: Static Analysis and Threshold Derivation (Vol. 1269, pp. 215–227). https://doi.org/10.1007/978-3-030-59155-7_16
- [19] Rama-cli/docs/metrics at master · restful-ma/rama-cli. (2020). GitHub. Retrieved August 9, 2026, from <https://github.com/restful-ma/rama-cli>
- [20] Decrop, A., Vandeloise, M., Heymans, P., & Perrouin, G. (2026). OASQuali: Automated quality analysis of OpenAPI specifications. Proceedings of the 26th International Conference on Web Engineering (ICWE 2026).